

Programación Orientada a Eventos

El DOM

Ahora que en el curso empezamos a trabajar con el lenguaje JAVASCRIPT, es momento de hablar de un punto fundamental que puede que hayas o no escuchado algún momento y es el DOM.

El DOM es la razón fundamental de por que utilizamos Javascript dentro del desarrollo web y si queremos visualizarlo, es la razón fundamental de como conectamos el HTML, con CSS y con JS.

Antes de hablar del DOM, debemos mencionar las estructuras de datos que debiste haber visto o conocido en cursos anteriores. Las estructuras de datos nos permiten modelar información de un modo que no solo estandariza, sino también nos permite aplicar ciertas estrategias o algoritmos particulares para un funcionamiento más eficiente.

Dentro de las estructuras de datos tenemos los árboles, que modelan datos a través de un nodo y hojas o hijos que se van derivando de su misma estructura.

A esto es donde nos lleva el DOM, si analizamos un poco observaremos que nuestro código HTML en realidad lo es una estructura de datos en forma de árbol, pues cada etiqueta puede contener información de otras etiquetas y cada una de estas pueden tener un número ilimitado de hijos. De la misma manera las propiedades se amarran como hojas a las etiquetas para dar valor particular según sea el caso.

Por lo tanto el DOM viene como significado de Document Object Model, que en español podríamos resolver como Modelo de Objetos del Documento. Este es una interfaz que permite crear, editar o eliminar elementos del documento. También se pueden agregar eventos para hacer dinámico el documento, pero esto lo veremos un poco más adelante.

La manera más simple visualiza al DOM como un árbol de tres nodos. Un nodo representa un documento HTML.

Echemos un vistazo a este código de HTML para entender mejor la estructura del DOM.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>HTML</title>
  </head>
  <body>
    <!-- Content -->
  </body>
</html>
```

El documento se llama nodo raíz y contiene un nodo hijo llamado **html**. El elemento **html** contiene dos hijos los cuales son los elementos **head** y **body**.

Ambos elementos **head** y **body** van a tener su propios hijos, en el caso del ejemplo veremos que **head** tiene como hijos a **meta** y a **title**.

Por tantos podemos acceder a estos elementos en el documento y hacer cambios a ellos usando Javascript.

¿Cómo seleccionar elementos en el documento?

Antes de empezar a ver métodos a diestra y siniestra, vamos a conocer la variable document, que desde el inicio está disponible para nosotros, y es esta variable de donde podemos partir para obtener todos los elementos del HTML.

Quizás un poco largo pero si hacemos un console.log() de document veremos que como resultados obtendremos literalmente todo nuestro HTML.

```
console.log(document)
```

Dentro de Javascript, vamos a encontrar varios métodos que nos permiten seleccionar un elemento del HTML dentro del document.

- **getElementById()**
- **querySelector()**
- **querySelectorAll**
- **createElement()**
- **appendChild()**

getElementById()

En HTML, los id se utilizan como identificadores únicos para los elementos HTML. Esto significa que no podemos tener el mismo nombre id para dos elementos diferentes. En ese sentido es lo mismo que en cualquier lenguaje de programación donde declaramos una variable y no podemos repetir el nombre de la misma.

Para ejemplificar el siguiente código sería incorrecto.

```
<p id="para">Soy un párrafo.</p>
<p id="para">Soy otro párrafo.</p>
```

Para que el **id** funcione, debemos segmentar o nombrar individualmente cada párrafo.

```
<p id="para1">Soy un párrafo.</p>
<p id="para2">Soy otro párrafo.</p>
```

Ya que hemos segmentado nuestros id ahora viene la forma de llamarlo desde Javascript.

```
document.getElementById("nombre de id va aquí")
```

Entonces el código final con el que podríamos llamar nuestro ejemplo sería el siguiente.

```
const paragraph1 = document.getElementById("para1");
console.log(paragraph1);
```

Si revisamos el resultado en la consola, veremos:

```
<p id="para1">Soy un párrafo.</p>
```

Si deseamos leer solamente el contenido del párrafo, entonces podemos usar la propiedad `textContent` dentro del `console.log()`.

```
const paragraph1 = document.getElementById("para1");
console.log(paragraph1.textContent);
```

El resultado sería:

```
Soy un párrafo.
```

querySelector()

El query selector nos va a permitir encontrar elementos con uno o más selectores de css.

A manera de ejemplo utilizaremos el siguiente:

```
<h1>Películas Famosas</h1>
<ul class="list">
  <li>Titanic</li>
  <li>Jurassic Park</li>
  <li>El señor de los Anillos</li>
  <li>Star Wars</li>
</ul>
```

Veremos que **querySelector()** nos permite obtener etiquetas generales.

```
const h1Element = document.querySelector("h1");
console.log(h1Element);
```

El resultado será nuestro **h1** declarado

```
<h1>Películas Famosas</h1>
```

Ahora bien, observa que en la lista hemos declarado una propiedad class con nombre **list**, con esto podemos obtener toda la etiqueta usando el **querySelector()**.

```
const list = document.querySelector(".list");
console.log(list);
```

El resultado en la consola será la etiqueta con todos sus elementos hijos.

```
<ul class="list">
  <li>Titanic</li>
  <li>Jurassic Park</li>
  <li>El señor de los Anillos</li>
  <li>Star Wars</li>
</ul>
```

Observa que al **querySelector()**, le estamos añadiendo un **.** y esto es por hacer referencia al nombre de la clase por tanto cuando llamemos a la clase debemos enviar como parámetro **.list** ó **{{nombre}}**.

querySelector() es un método genérico el cual permite obtener tanto etiquetas generales, clases o igual que con **getElementById** identificadores. Para poder llamar un id deberás utilizar en vez del **.** el símbolo de **#**.

querySelectorAll()

El query selector all encuentra todos los métodos que coinciden con el selector de css y devuelve una lista de todos esos nodos. Si volvemos con nuestro ejemplo anterior de:

```
<ul class="list">
  <li>Titanic</li>
  <li>Jurassic Park</li>
  <li>El señor de los Anillos</li>
  <li>Star Wars</li>
</ul>
```

Si quisiéramos encontrar todos los elementos **li** de nuestro ejemplo, podríamos utilizar el combinado de hijos **>** para encontrar todos los elementos hijos de **ul**.

```
const listItems = document.querySelectorAll("ul > li");
console.log(listItems);
```

Aquí veremos un resultado un poco diferente a los anteriores, si exploras tu navegador incluso verás más detalles. En este caso estas viendo todas las propiedades de los nodos hijos **li** .

```
{
  "0": {},
  "1": {},
  "2": {},
  "3": {}
}
```

Como query selector obtiene todo, veremos que cuando hay varios hijos se modifica la forma de visualizar los datos. Si queremos ir elemento hijo por elemento hijo vamos a necesitar iterar el resultado. Para ello podemos hacer lo siguiente.

```
const listaDeElementos = document.querySelectorAll("ul > li");
```

```
listaDeElementos.forEach((item) => {
  console.log(item);
});
```

```
<li>Titanic</li>
<li>Jurassic Park</li>
<li>El señor de los Anillos</li>
<li>Star Wars</li>
```

El resultado será ahora sí la etiqueta con su valor interno.

createElement()

También podemos utilizar **createElement()** para agregar nuevos elementos al DOM.

Veamos el siguiente ejemplo añadiendo la etiqueta:

```
<h2>Elementos que se utilizan en desarrollo Web:</h2>
```

Con esta nueva etiqueta **h2** queremos agregar una lista de elementos en la parte inferior.

Primero vamos a crear un elemento **ul** usando **document.createElement()**. Asignaremos eso a una variable llamada listaSinOrden.

```
let listaSinOrden = document.createElement("ul");
```

Después necesitaremos añadir el elemento usando el método **appendChild()**.

```
document.body.appendChild(listaSinOrden);
```

Antes de avanzar observa como estamos haciendo para anexar directamente el **ul** después de de nuestra etiqueta **h2**, y esto lo hacemos sin la necesidad de hacer referencia al **h2** como tal.

Para hacerlo accedemos al **document** y desde aquí podemos acceder a la etiqueta **body**, al utilizar **appendChild()** estamos anexando el elemento al final del elemento seleccionado, en este caso al final del **body** y por tanto adelante de nuestra etiqueta **h2**.

Ahora debemos agregar varios elementos **li** dentro del elemento **ul** usando nuevamente **createElement()**

```
let elemento1Lista = document.createElement("li");
let elemento2Lista = document.createElement("li");
let elemento3Lista = document.createElement("li");
```

Ojo: Aquí solo estamos creando las etiquetas

Después podemos utilizar la propiedad **textContent** para agregar texto para nuestros 3 elementos de la lista.

```
let elemento1Lista = document.createElement("li");
elemento1Lista.textContent = "HTML";
let elemento2Lista = document.createElement("li");
elemento2Lista.textContent = "CSS";
let elemento3Lista = document.createElement("li");
elemento3Lista.textContent = "JS";
```

El último paso es agregar el método **appendChild()** para que los elementos de la lista sean agregados al **ul**.

```
let elemento1Lista = document.createElement("li");
elemento1Lista.textContent = "HTML";
listaSinOrden.appendChild(elemento1Lista);
let elemento2Lista = document.createElement("li");
elemento2Lista.textContent = "CSS";
listaSinOrden.appendChild(elemento2Lista);
let elemento3Lista = document.createElement("li");
elemento3Lista.textContent = "JS";
listaSinOrden.appendChild(elemento3Lista);
```

¿Cómo utilizar style para modificar CSS?

Ya hemos modificado el HTML, pero la ventaja de utilizar el DOM, es que como ya mencioné nos da acceso a como se encuentra en ese momento el HTML incluyendo sus propiedades. Por lo mismo nos da acceso al estado del CSS de las etiquetas.

Nota: Recuerda el poder modificar CSS, no nos da acceso directo a crear estilos dentro de los archivos externos CSS. Sin embargo en usos más avanzados podemos crear etiquetas de style y agregarlas al DOM, aunque esto no es una buena práctica no hay una limitante sintáctica que nos impida hacerlo.

En este ejemplo vamos a cambiar el color de texto de un **h3** de negro a azul usando la propiedad **style**. En vez de agregar directamente el **html** vamos a añadirlo de manera dinámica como vimos en el paso anterior.

```
let newH3 = document.createElement("h3");
newH3.textContent = "Hola soy un texto en color negro"
document.body.appendChild(newH3);
```

Después usamos el **querySelector()** para obtener el **h3**.

```
const h3 = document.querySelector("h3");
```

Ahora vamos a utilizar **h3.style.color** para modificar el color, nota que al acceder a la etiqueta podemos acceder a su propiedad **style**, y desde aquí podemos aplicar el **css** que necesitemos.

```
h3.style.color = "blue";
```

Por último quizás el texto del **h3** ya no sea tan representativo, que tal si lo modificamos como ya hemos visto anteriormente.

```
h3.textContent = "Ohh no me cambiaron a azul."
```

Si observas el resultado en el navegador verás que el cambio se realizó correctamente. Pero si somos curiosos que pasarías si imprimimos en consola el valor de **newH3**.

```
console.log(newH3)
```

Verás que de la misma manera **newH3** se actualiza con el nuevo valor, y esto es por que los valores no se guardan de manera estática sino por referencia unos de otros. En el día a día no vamos a trabajar de esta manera, pero es un buen ejemplo para entender como se modifican las cosas.

Al final verás que en cuanto al **style** podrás modificar prácticamente cualquier propiedad de css que necesites, desde background-color, border-style, font-size, etc.

Usar eventos con Javascript

Los eventos son cosas que pasan en el documento que estás programando, el cual se encarga de avisarte para que tu código pueda hacer algo al respecto.

Un ejemplo simple sería, si el usuario hace un clic en un botón de la página lo normal es reaccionar a esa acción y realizar algo a partir de ello.

Existen diversos tipos de eventos, y depende más del desarrollador identificar que eventos quiere obtener del usuario. Algunas ideas que puedes obtener para ello son las siguientes:

- El usuario selecciona, hace clic o pasar el ratón por encima de cierto elemento.
- El usuario presiona una tecla del teclado.
- El usuario redimensiona o cierra la ventana del navegador.
- Una página web terminó de cargarse.
- Un formulario fue enviado.
- Un vídeo se reproduce, se pausa o termina.
- Ocurrió un error.

Para este ejemplo agregaremos el siguiente botón, nuevamente de manera dinámica.

```
let newButton = document.createElement("button");
newButton.textContent = "Mostrar Alerta"
newButton.setAttribute("id", "btn");
document.body.appendChild(newButton);
```

Si bien estamos utilizando el código de los ejemplos anteriores para agregar un **button** observa que utilizamos el método **setAttribute** para agregar el **id** a nuestro botón, esto sería el equivalente a que en el HTML declaráramos la etiqueta de la siguiente manera:

```
<button id="btn">Mostrar Alerta</button>
```

Ahora solo para practicar vamos a obtener el botón utilizando su ID.

```
const button = document.getElementById("btn");
```

Para poder agregar manejo de eventos a cualquier etiqueta del HTML o dicho de otra forma, para poder agregar eventos a cualquier elemento del DOM, utilizaremos el método **addEventListener()**. Este método asociará un evento a la lista de eventos estandarizados que se tienen para los navegadores, si tienes duda en que tipo de eventos se pueden utilizar checa la siguiente [página](#), aquí verás el listado completo.

Como nosotros queremos asociar un **click** a nuestro botón, deberemos hacer lo siguiente.

```
button.addEventListener("click", () => {
  alert("Gracias por el click");
});
```

El resultado es que al hacer clic en nuestro botón se despliega una alerta mostrando el mensaje. Como puedes concluir, realizar acciones desde Javascript lo es todo para hacer páginas como las que existen hoy en día. Siguiendo los principios de diseño, manteniendo los estándares y siendo simple al momento de hacer las cosas podemos tener desarrollo complejos y elegantes que sean dignos de premios.

Si quieres ahondar más en el tema te recomiendo el siguiente [artículo](#) el cual crea varios proyectos de Javascript paso a paso para ir entendiendo el potencial que tiene, no olvides que la práctica es la única manera de entender como funciona y sin ello te estarás quedando atrás.

Ver ejemplo completo